

使用檢索增強生成功能建構代理

來源：<https://codelabs.developers.google.com/codelabs/production-ready-ai-with-gc/7-advanced-agent-capabilities/building-agents-with-retrieval-augmented-generation?hl=zh-tw>

步驟數：13

「瞭解如何在 Google Cloud 中，使用 Vertex AI Search 和 Agent Development Kit，開發端對端 Agentic RAG 應用程式。」

目錄

1. 簡介
2. 瞭解檢索增強生成
3. 從 RAG 管道到代理式 RAG
4. 專案設定
5. 開啟 Cloud Shell 編輯器
6. 啟用 API
7. 設定環境
8. 建立 Vertex AI Search 資料儲存庫
9. 建立結構化資料的自訂工具
10. 建構及執行 RAG 代理程式
11. 測試代理
12. 為工作選擇服務
13. 結論

1. 簡介

總覽

本實驗室的目標是瞭解如何在 Google Cloud 開發端對端代理程式檢索增強生成 (RAG) 應用程式。在本實驗室中，您將建構財務分析代理程式，結合兩種不同來源的資訊來回答問題：非結構化文件 ([Alphabet 的季度 SEC 申報資料](#) - 美國每間上市公司提交給證券交易委員會的財務報表和營運詳細資料)，以及結構化資料 (歷史股價)。

您將使用 [Vertex AI Search](#)，為非結構化財務報表建構強大的語意搜尋引擎。針對結構化資料，您將建立自訂 Python 工具。最後，您將使用 [Agent Development Kit \(ADK\)](#) 建構智慧型代理，這個代理能推論使用者的查詢內容、決定要使用的工具，並將資訊整合為連貫的回覆。

學習內容

- 設定 Vertex AI Search 資料儲存庫，對私人文件執行語意搜尋。
- 建立自訂 Python 函式，做為代理的工具。
- 使用 Agent Development Kit (ADK) 建構多工具代理。
- 結合非結構化和結構化資料來源的擷取結果，回答複雜問題。
- 測試並與展現推理能力的代理程式互動。

課程內容

本實驗室的學習內容包括：

- 檢索增強生成 (RAG) 和代理程式 RAG 的核心概念。
 - 如何使用 Vertex AI Search，對文件執行語意搜尋。
 - 如何建立自訂工具，向代理程式公開結構化資料。
 - 如何使用 Agent Development Kit (ADK) 建構及調度管理多工具代理。
 - 瞭解代理程式如何運用推論和規劃功能，使用多個資料來源回答複雜問題。
-

2. 瞭解檢索增強生成

大型生成模型 (簡稱大型語言模型或 LLM、視覺語言模型等) 功能強大，但有其固有的限制。這類模型在[預先訓練](#)時會凍結知識，因此知識是靜態的，很快就會過時。即使經過[微調](#)，模型知識也不會變得更貼近現況，因為這並非訓練後階段的目標。

大型語言模型 (尤其是「思考」模型) 的訓練方式，是「獎勵」模型提供某種答案，即使模型本身沒有可支援該答案的事實資訊。也就是說，模型會「產生幻覺」，自信地生成看似合理，但事實上不正確的資訊。

檢索增強生成 (RAG) 是一種強大的架構模式，專門用來解決這些問題。這是一種架構框架，可將大型語言模型即時連結至外部權威知識來源，藉此提升模型功能。RAG 系統中的 LLM 不會只依賴靜態的預先訓練知識，而是會先擷取與使用者查詢相關的資訊，然後使用這些資訊生成更準確、及時且符合情境的回覆。

這項方法直接解決了生成模型最顯著的缺點：知識固定在某個時間點，且容易生成不正確的資訊或「幻覺」。RAG 就像是讓 LLM 「開書考」，而「書」就是您私有的特定領域最新資料。為 LLM 提供事實脈絡的程序稱為「[建立基準](#)」。

RAG 的 3 個步驟

標準的檢索增強生成程序可分為三個簡單步驟：

1. **擷取**：使用者提交查詢時，系統會先搜尋外部知識庫 (例如文件存放區、資料庫或網站)，找出與查詢相關的資訊。
2. **增強**：接著，系統會將檢索到的資訊與原始使用者查詢合併，形成擴充提示。這項技術有時稱為「提示填充」，因為它會以事實脈絡豐富提示。
3. **生成**：將擴充提示詞提供給 LLM，然後生成回覆。由於模型已取得相關的真實資料，因此輸出內容「有依據」，不太可能不準確或過時。

RAG 的優點

RAG 架構的推出，徹底改變了實用且值得信賴的 AI 應用程式建構方式。主要福利包括：

- **提升準確率並減少幻覺**：RAG 會根據可驗證的外部事實生成回覆，大幅降低 LLM 編造資訊的風險。
- **存取最新資訊**：RAG 系統可連結至持續更新的知識庫，根據最新資訊提供回覆，這是經過靜態訓練的 LLM 無法做到的事。
- **提高使用者信任度和透明度**：LLM 的回覆內容是以擷取的文件為依據，因此系統可以提供來源的引用和連結。使用者可自行驗證資訊，對應用程式更有信心。
- **成本效益**：使用新資料持續微調或重新訓練 LLM，需要耗費大量運算資源和資金。使用 RAG 時，更新模型知識就像更新外部資料來源一樣簡單，效率更高。
- **網域專業化和隱私權**：RAG 可讓個人和機構在查詢時，向 LLM 提供私密專屬資料，不必將這類私密資料納入模型的訓練集。這項技術可支援強大的網域專屬應用程式，同時維護資料隱私和安全。

擷取

「檢索」步驟是所有 RAG 系統的核心。系統擷取資訊的品質和關聯性，會直接決定最終生成答案的品質和關聯性。有效的 RAG 應用程式通常需要使用各種技術，從不同類型的資料來源擷取資訊。主要檢索方法分為三類：以關鍵字為準、語意和結構化。

從非結構化資料擷取資訊

從歷史來看，非結構化資料擷取是傳統搜尋的別稱。這項技術歷經多次轉換，您可從兩種主要做法中獲益。

語意搜尋是您可以在 Google Cloud 中大規模執行的最有效率技術，可提供最先進的效能和高度控制權。

- **以關鍵字為基礎 (詞彙) 的搜尋**：這是傳統的搜尋方法，可追溯至 1970 年代最早的資訊檢索系統。詞彙搜尋的運作方式是將使用者查詢中的字詞 (或「權杖」) 與知識庫文件中的完全相同字詞進行比對。如果查詢需要精確的特定字詞 (例如產品代碼、法律條款或專有名稱)，這項功能就非常實用。
- **語意搜尋**：語意搜尋或「有意義的搜尋」是一種較為現代的方法，旨在瞭解使用者的意圖和查詢的情境意義，而不只是字面上的關鍵字。現代語意搜尋技術採用**嵌入**，這是一種機器學習技術，可將複雜的高維度資料對應至數值向量的低維度向量空間。這些向量的設計方式，是讓意義相似的文字在向量空間中彼此靠近。系統會將「適合家庭的最佳犬種有哪些？」的搜尋查詢轉換為向量，然後在該空間中搜尋與其「最鄰近」的文件向量。因此，即使文件中沒有「狗」這個確切字詞，系統也能找出提及「黃金獵犬」或「友善犬隻」的文件。近似最鄰近 (ANN) 演算法可有效執行這類高維度搜尋。ANN 演算法不會將查詢向量與每個文件向量進行比較 (這對大型資料集來說速度太慢)，而是使用智慧型索引結構，快速找出最接近的向量。

從結構化資料擷取

並非所有重要知識都儲存在非結構化文件中。通常最精確且有價值的資訊會以結構化格式呈現，例如關聯式資料庫、NoSQL 資料庫，或是某種 API，例如天氣資料或股價的 REST API。

從結構化資料中擷取資訊通常比搜尋非結構化文字更直接準確。語言模型不必搜尋語意相似性，而是能夠制定及執行精確的查詢，例如資料庫的 SQL 查詢，或是特定地點和日期的天氣 API 呼叫。

這項技術與 AI 代理採用的**函式呼叫**相同，可讓語言模型以確定性的結構化方式，與可執行的程式碼和外部系統互動。

3. 從 RAG 管道到代理式 RAG

隨著 RAG 概念本身不斷演進，實作架構也隨之發展。原本簡單的線性管道，現在已發展成由 AI 代理協調的動態智慧系統。

- **簡單 (或原始) RAG**：這是我們目前討論的基礎架構，也就是檢索、擴增和生成這三個步驟的線性程序。這項技術屬於反應式，會為每個查詢採用固定路徑，非常適合簡單的問答工作。
- **進階 RAG**：這是指管道的演進，在管道中加入額外步驟，以提升檢索到的內容品質。這些強化作業可以在擷取步驟前後進行。
 - **檢索前**：可使用查詢重寫或擴充等技術。系統可能會分析初始查詢，並重新措辭，以便檢索系統更有效率地擷取資訊。
 - **檢索後**：檢索出初始文件集後，系統會套用重新排名模型，根據關聯性為文件評分，並將最佳文件置於頂端。這在混合型搜尋中尤其重要。擷取後，另一個步驟是篩選或壓縮擷取的背景資訊，確保只有最顯著的資訊會傳送給 LLM。
- **代理程式 RAG**：這是 RAG 架構的尖端技術，代表從固定管道到自主智慧程序的典範轉移。在代理式 RAG 系統中，整個工作流程是由一或多個 **AI 代理** 管理，這些代理可以推論、規劃及動態選擇要執行的動作。

如要瞭解 Agentic RAG，必須先瞭解 AI 代理程式的構成要素。代理不只是大型語言模型，這項系統包含多個重要元件：

1. **將 LLM 做為推理引擎**：代理程式不僅會使用 Gemini 等功能強大的 LLM 生成文字，還會將其做為中央「大腦」，負責規劃、決策及分解複雜工作。
2. **工具組**：代理可存取一系列功能，並決定要使用哪些功能來達成目標。這些工具可以是計算機、網頁搜尋 API、傳送電子郵件的功能，或對本實驗室而言最重要的**各種知識庫的檢索器**。
3. **記憶**：設計代理時，可以同時加入短期記憶 (記住當前對話的脈絡) 和長期記憶 (回顧過往互動資料)，以便提供更個人化且連貫的體驗。
4. **規劃和反思**：最先進的代理程式會展現複雜的推理模式。這類模型可以接收複雜的目標，並制定多步驟計畫來達成目標。然後執行這項計畫，甚至**反思**行動結果、找出錯誤，並自行修正做法，以改善最終結果。

Agentic RAG 是一項顛覆性技術，因為它導入了靜態管道所沒有的自主性和智慧層。

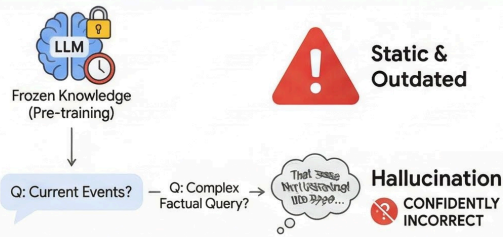
- **彈性和適應性**：代理程式不會受限於單一檢索路徑。根據使用者查詢，判斷最佳資訊來源。在單一使用者要求的情境下，Gemini 可能會決定先查詢結構化資料庫，然後對非結構化文件執行語意搜尋，如果仍找不到答案，則使用 Google 搜尋工具在公開網路上尋找。
- **複雜的多步驟推論**：這項架構擅長處理複雜查詢，這類查詢需要多個連續的擷取和處理步驟。

以「找出克里斯多福諾蘭執導的前 3 大科幻電影，並簡要說明劇情」為例，簡單的 RAG 管道會失敗。

但代理程式可以將其分解為：

1. **計畫**：首先，我需要找到電影。然後，我需要找出每部電影的劇情。
2. **動作 1**：使用結構化資料工具查詢諾蘭的科幻電影資料庫，找出評分最高的前 3 部電影，並依評分由高至低排序。
3. **觀察 1**：工具傳回「全面啟動」、「星際效應」和「天能」。
4. **動作 2**：使用非結構化資料工具 (語意搜尋) 尋找「全面啟動」的劇情。
5. **觀察 2**：系統會擷取劇情。
6. **步驟 3**：對「星際效應」重複上述步驟。
7. **步驟 4**：對「天能」重複上述步驟。
8. **最終統整**：將所有擷取的資訊統整成單一連貫的答案，提供給使用者。

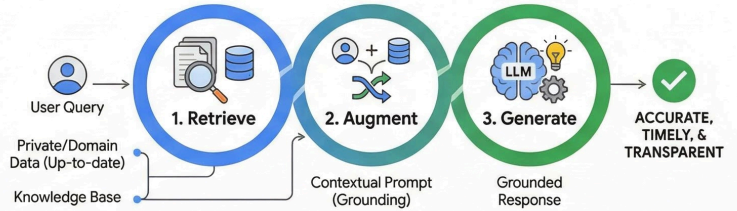
The Problem: Generative Model Limitations



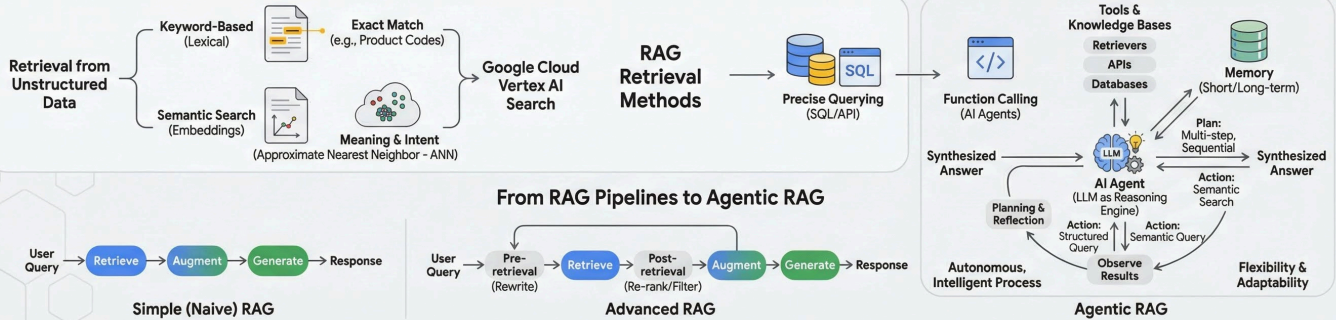
Risk: Fabricated Information

The Solution: Retrieval-Augmented Generation (RAG)

3 Steps of RAG



Benefits: Enhanced Accuracy, Current Info, User Trust, Cost-Effective, Privacy



From RAG Pipelines to Agentic RAG



步驟 4 · 約 0 分鐘

4. 專案設定

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

Google 帳戶

如果沒有個人 Google 帳戶，請[建立 Google 帳戶](#)。

請使用個人帳戶，而非公司或學校帳戶。

公司和學校帳戶可能設有限制，導致您無法啟用本實驗室所需的 API。

登入 Google Cloud 控制台

使用個人 Google 帳戶登入 [Google Cloud 控制台](#)。

啟用計費功能

使用試用帳單帳戶 (選用)

如要參加這個研討會，您需要有具備抵免額的帳單帳戶。請使用本程式碼研究室頂端橫幅中的抵免額開始操作。如果已連結帳單帳戶，可以略過這個步驟。

設定個人帳單帳戶

如果使用 Google Cloud 抵免額設定計費，可以略過這個步驟。

如要設定個人帳單帳戶，請[前往這裡在 Cloud 控制台中啟用帳單](#)。

注意事項：

- 完成本實驗室的 Cloud 資源費用應不到 \$1 美元。
- 您可以按照本實驗室結尾的步驟刪除資源，以免產生後續費用。
- 新使用者可獲得價值 [\\$300 美元的免費試用期](#)。

建立專案 (選用)

如果沒有要用於本實驗室的現有專案，請[在這裡建立新專案](#)。

步驟 5 · 約 0 分鐘

5. 開啟 Cloud Shell 編輯器

1. 按一下這個連結，直接前往 [Cloud Shell 編輯器](#)
2. 如果系統在今天任何時間提示您授權，請點選「授權」繼續操作。

Authorize Cloud Shell

gcloud is requesting your credentials to make a GCP API call.

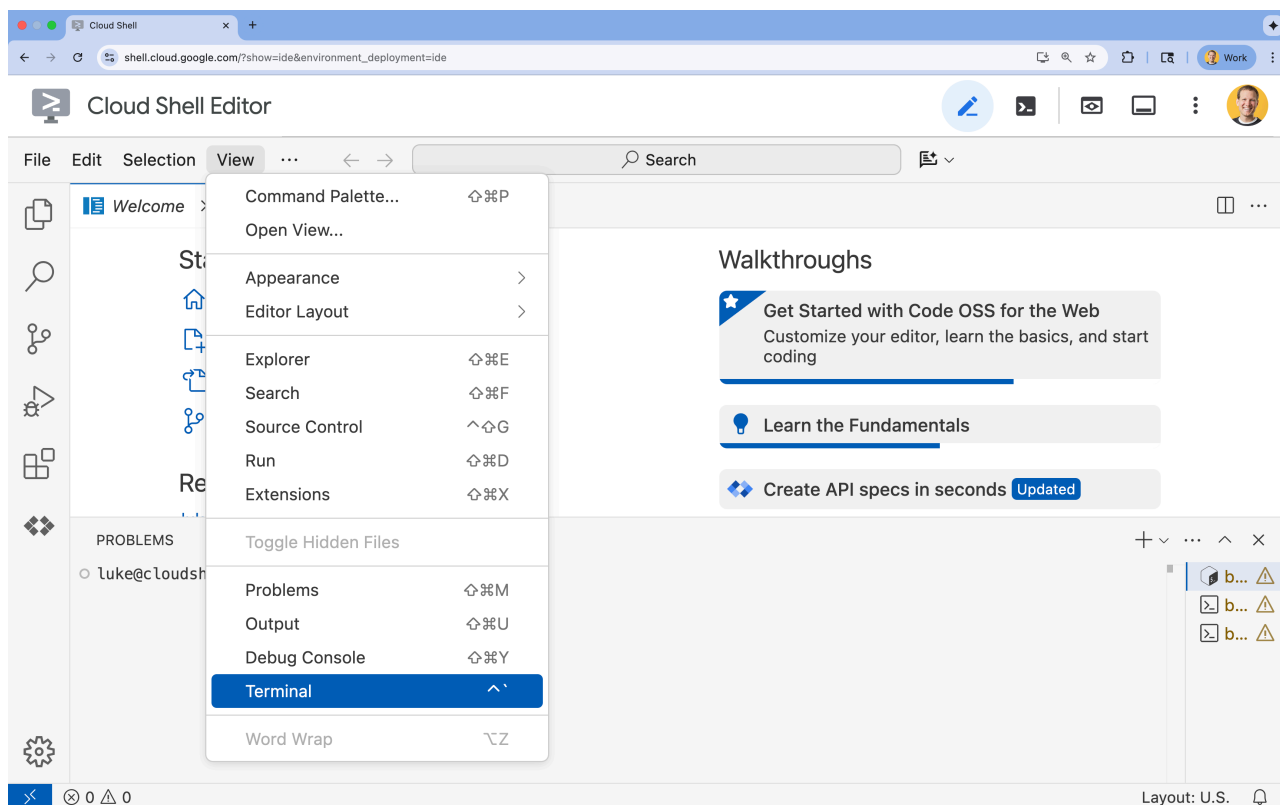
Click to authorize this and future calls that require your credentials.

REJECT

AUTHORIZE

3. 如果畫面底部未顯示終端機，請開啟終端機：

- 按一下「查看」
- 按一下「終端機」



4. 在終端機中，使用下列指令設定專案：

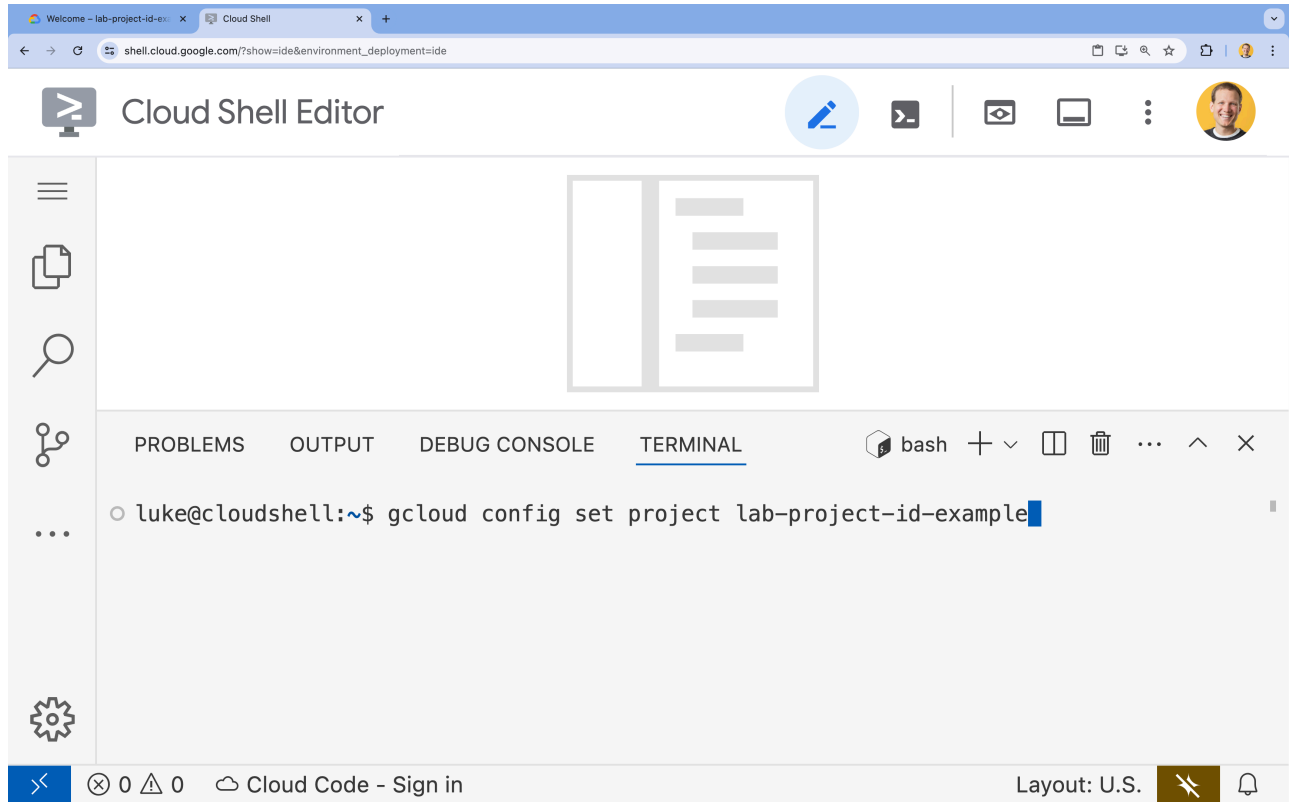
```
gcloud config set project [PROJECT_ID]
```

- 範例：

```
gcloud config set project lab-project-id-example
```

- 如果忘記專案 ID，可以使用下列指令列出所有專案 ID：

```
gcloud projects list
```



5. 您應會看到下列訊息：

```
Updated property [core/project].
```

如果看到 **WARNING** 並收到 **Do you want to continue (Y/n)?** 提示，表示您可能輸入了錯誤的專案 ID。按下 **n** 和 **Enter**，然後再次嘗試執行 **gcloud config set project** 指令。

步驟 6 · 約 0 分鐘

6. 啟用 API

如要使用 [Agent Development Kit](#) 和 Vertex AI Search，請務必在 Google 雲端專案中啟用必要的 API。

1. 在終端機中啟用 API：

```
gcloud services enable \  
  aiplatform.googleapis.com \  
  discoveryengine.googleapis.com
```

API 簡介

- **Vertex AI API** ([aiplatform.googleapis.com](#)) 可讓代理程式與 Gemini 模型通訊，進行推論和生成作業。
 - **Discovery Engine API** ([discoveryengine.googleapis.com](#)) 可為 Vertex AI Search 提供支援，讓您建立資料儲存庫，並對非結構化文件執行語意搜尋。
-

步驟 7 · 約 0 分鐘

7. 設定環境

開始編寫 AI 代理程式碼前，您需要準備開發環境、安裝必要程式庫，並建立所需資料檔案。

建立虛擬環境並安裝依附元件

1. 為代理建立目錄，然後前往該目錄。在終端機中執行下列程式碼：

```
mkdir financial_agent
cd financial_agent
```

2. 建立虛擬環境：

```
uv venv --python 3.12
```

3. 啟動虛擬環境：

```
source .venv/bin/activate
```

4. 安裝 Agent Development Kit (ADK) 和 pandas。

```
uv pip install google-adk pandas
```

建立股價資料

由於實驗室需要特定的歷史股票資料，才能展示代理程式使用結構化工具的能力，因此您將建立包含這類資料的 CSV 檔案。

1. 在 `financial_agent` 目錄中，於終端機執行下列指令，建立 `goog.csv` 檔案：

```
cat <<EOF > goog.csv
Date,Open,High,Low,Close,Adj Close,Volume
2023-01-03,89.830002,91.550003,89.019997,89.699997,89.699997,20738500
2023-03-31,101.440002,103.889999,101.040001,103.730003,103.730003,36823200
2023-06-30,120.870003,122.029999,120.300003,120.970001,120.970001,23824700
2023-09-29,134.080002,134.550003,131.320007,131.850006,131.850006,21124200
2025-07-10,185.000000,188.000000,184.500000,186.500000,186.500000,25000000
EOF
```

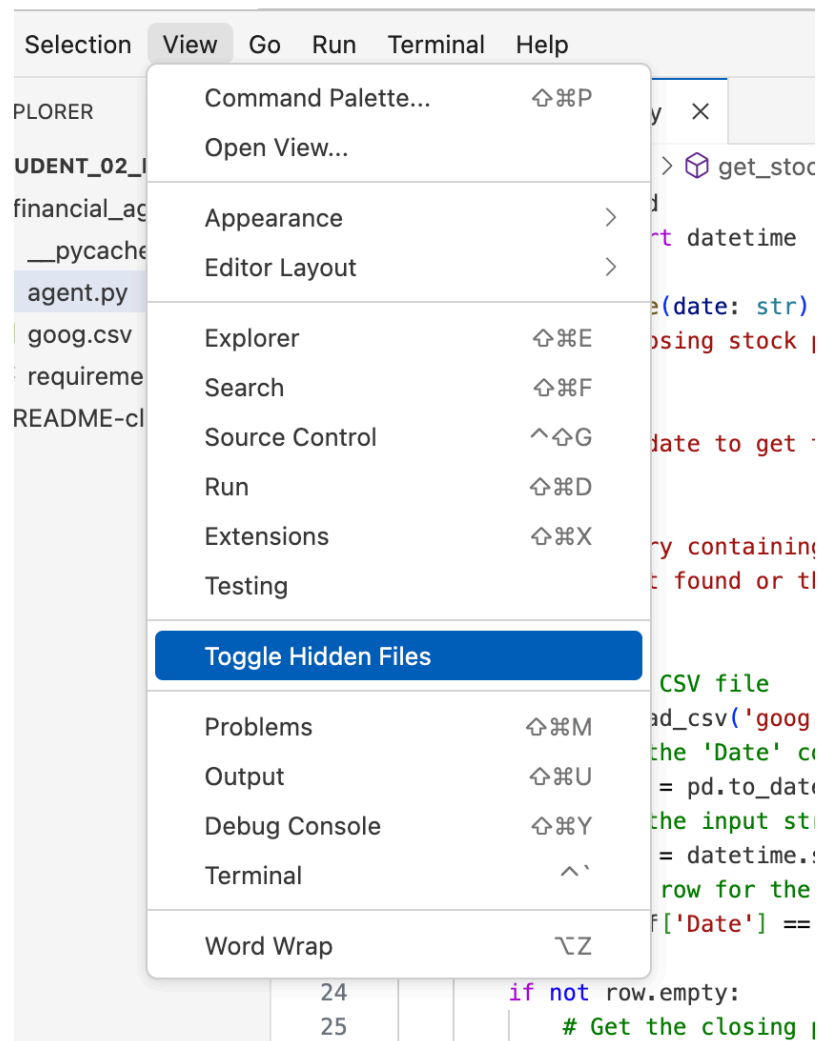
設定環境變數

2. 在 `financial_agent` 目錄中，建立 `.env` 檔案來設定代理程式的環境變數。這會告知 ADK 要使用哪個專案、位置和模型。在終端機中執行下列程式碼：

```
# Create the .env file using the captured variables
cat << EOF > .env
GOOGLE_GENAI_USE_VERTEXAI=TRUE
GOOGLE_CLOUD_PROJECT="${GOOGLE_CLOUD_PROJECT}"
GOOGLE_CLOUD_LOCATION="us-central1"
EOF
```

注意：在實驗室後續步驟中，如果需要修改 `.env` 檔案，但 `financial_agent` 目錄中沒有顯示該檔案，請使用「View / Toggle Hidden Files」選單項目，在 Cloud Shell 編輯器中切換隱藏檔案的顯示狀態。

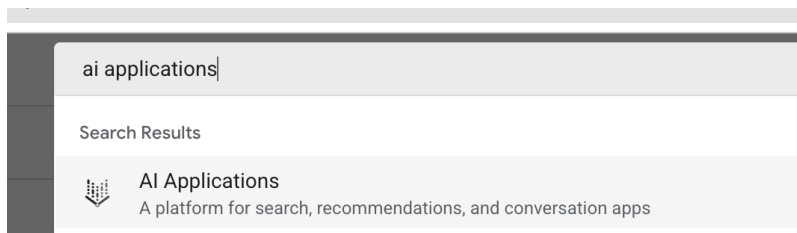
Cloud Shell Editor



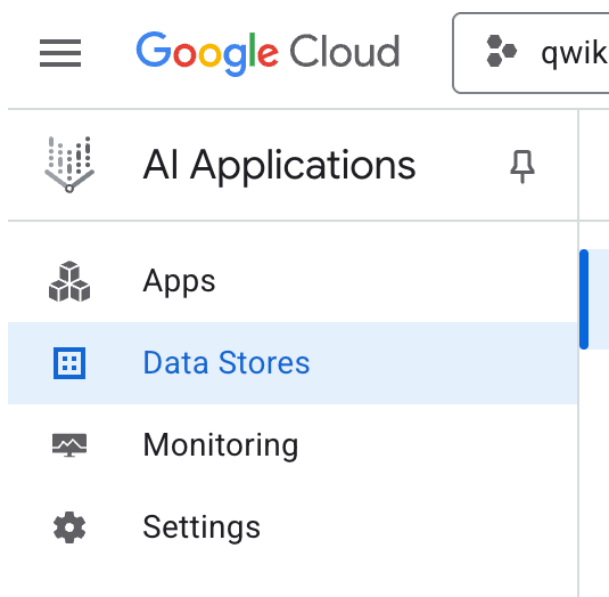
8. 建立 Vertex AI Search 資料儲存庫

如要讓代理程式回答有關 Alphabet 財務報表的問題，請建立 Vertex AI Search 資料儲存庫，其中包含 Alphabet 的公開 SEC 備案文件。

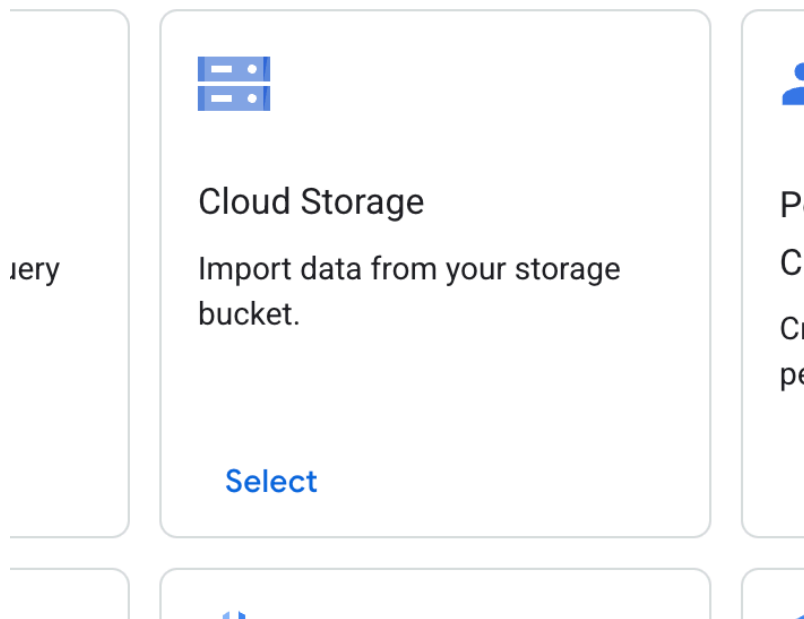
1. 在新瀏覽器分頁中開啟 [Cloud 控制台](https://console.cloud.google.com) (console.cloud.google.com)，然後使用頂端的搜尋列前往「AI 應用程式」。



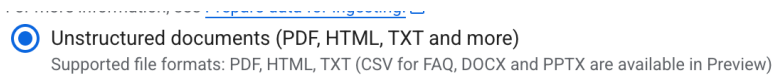
2. 如果系統提示，請勾選條款及細則核取方塊，然後按一下「繼續並啟用 API」。
3. 選取左側導覽選單中的「資料儲存庫」。



4. 點選「+ 建立資料儲存庫」。
5. 找到「Cloud Storage」資訊卡，然後點按「選取」。



6. 在資料來源部分，選取「非結構化文件」。



7. 在匯入來源 (選取要匯入的資料夾或檔案) 中，輸入 Google Cloud Storage 路徑 `cloud-samples-data/gen-app-builder/search/alphabet-sec-filings`。

8. 按一下「繼續」。

9. 將位置設為「全域」。

10. 在「資料儲存庫名稱」中輸入

`alphabet-sec-filings`

11. 展開「文件處理選項」部分。

▼ Document processing options

12. 在「Default document parser」(預設文件剖析器) 下拉式清單中，選取「Layout Parser」(版面配置剖析器)。

Document parsing

Select a parser to apply to your documents. [Learn more](#)

Default document parser *

- Digital Parser
- OCR Parser
- Layout Parser
- + add exception for a file format

Layout parser settings

13. 在「版面配置剖析器設定」選項中，選取「啟用表格註解」和「啟用圖片註解」。
14. 按一下「繼續」。
15. 選取「一般定價」做為定價模式 (即付即用、依用量計費)，然後按一下「建立」。

注意：為進行本實驗室，請「不要」選取「可設定的價格」選項。

16. 資料儲存庫會開始匯入文件。

The screenshot shows the 'AI Applications' sidebar on the left with 'Data Stores' selected. The main panel is titled 'Data Stores' and includes a 'Create Data Store' button. Below this is a 'Filter' input field. A table lists data stores, with 'alphabet-sec-filings' highlighted by a red box.

Name
alphabet-sec-filings

17. 按一下資料儲存庫名稱，然後從「資料儲存庫」表格複製其 ID。在下一步中將會用到。

The screenshot shows the 'Data stores' page for the 'alphabet-sec-filings' data store. The breadcrumb navigation is 'Data stores > alphabet-sec-filings > Data'. The data store name 'alphabet-sec-filings' is at the top. Below is a table with details about the data store, with the 'Data store ID' highlighted by a red box.

alphabet-sec-filings	
Data store ID	goog-docs_1752441485216
Type	Unstructured data
Serving state	Enabled
Region	global
Language	N/A

18. 在 Cloud Shell 編輯器中開啟 `.env` 檔案，然後將資料儲存庫 ID 附加為 `DATA_STORE_ID="YOUR_DATA_STORE_ID"` (將 `YOUR_DATA_STORE_ID` 替換成上一個步驟中的實際 ID)。注意：系統需要幾分鐘的時間，才能在資料儲存庫中匯

入、剖析及建立資料索引。如要查看處理程序，請點選資料儲存庫名稱開啟其屬性，然後開啟「活動」分頁標籤。等待狀態變為「匯入完成」。

Activity

[+ Import data](#)

 Filter

Status	Detail	Items succeeded	Operation na
 Import completed	No errors	78	import-docurr

步驟 9 · 約 0 分鐘

9. 建立結構化資料的自訂工具

接著，您將建立 Python 函式，做為代理的工具。這個工具會讀取 `goog.csv` 檔案，擷取特定日期的過往股價。

1. 在 `financial_agent` 目錄中，建立名為 `agent.py` 的新檔案。在終端機中執行下列指令：

```
cloudshell edit agent.py
```

2. 在 `agent.py` 中加入下列 Python 程式碼。這段程式碼會匯入相依項目，並定義 `get_stock_price` 函式。

```

from datetime import datetime
import os
import logging

import google.cloud.logging
from google.adk.tools import VertexAiSearchTool
from google.adk.tools.agent_tool import AgentTool
from google.adk.agents import LlmAgent

import pandas as pd

def get_stock_price(date: str) -> dict:
    """Gets the closing stock price for a given date.

    Args:
        date: The date to get the stock price for, in YYYY-MM-DD format.

    Returns:
        A dictionary containing the closing price, or an error message if the
        date is not found or the format is incorrect.
    """
    try:
        # Load the CSV file
        df = pd.read_csv('goog.csv')
        # Convert the 'Date' column to datetime objects
        df['Date'] = pd.to_datetime(df['Date'])
        # Convert the input string to a datetime object
        query_date = datetime.strptime(date, '%Y-%m-%d')
        # Find the row for the given date
        row = df[df['Date'] == query_date]

        if not row.empty:
            # Get the closing price
            close_price = row['Close'].iloc[0]
            return {"status": "success", "date": date, "closing_price": close_price}
        else:
            return {"status": "error", "message": f"No data found for date: {date}"}
    except FileNotFoundError:
        return {"status": "error", "message": "Stock data file (goog.csv) not found."}
    except Exception as e:
        return {"status": "error", "message": f"An error occurred: {str(e)}"}

```

請注意函式的詳細說明字串。說明函式的作用、參數 (Args) 和傳回值 (Returns)。ADK 會使用這個 docstring，教導代理如何及何時使用這項工具。

10. 建構及執行 RAG 代理程式

現在可以組裝代理程式了。您會將非結構化資料的 Vertex AI Search 工具，與結構化資料的自訂 `get_stock_price` 工具結合。

1. 在 `agent.py` 檔案中附加下列程式碼。這段程式碼會匯入必要的 ADK 類別、建立工具例項，並定義代理程式。

```
logging.basicConfig(level=logging.INFO)
cloud_logging_client = google.cloud.logging.Client()
cloud_logging_client.setup_logging()

# 1. Create the Vertex AI Search tool
full_datastore_id =
f"projects/{os.environ['GOOGLE_CLOUD_PROJECT']}/locations/global/collections/default_coll
ection/dataStores/{os.environ['DATA_STORE_ID']}"
vertex_ai_search_tool = VertexAiSearchTool(
    data_store_id=full_datastore_id
)

# 2. Define the Search+Q&A agent
# Since we cannot combine tools with other tools in a single agent,
# we create a separate Search+Q&A agent which will be used as a tool by the main root
agent.

doc_qa_agent = LlmAgent(
    name="search_and_qna_agent",
    model="gemini-2.5-flash",
    tools=[vertex_ai_search_tool],
    instruction="""You are a helpful assistant that answers questions based on
information found in the document store.
Use the search tool to find relevant information before answering.
If the answer isn't in the documents, say that you couldn't find the information.
""",
    description="Answers questions using a specific Vertex AI Search datastore.",
)

# 3. Define the root agent with 2 tools.

root_agent = LlmAgent(
    name="financial_agent",
    model="gemini-2.5-flash",
    tools=[
        AgentTool(doc_qa_agent),
        get_stock_price,
    ],
    instruction="""You are an Financial Analytics Agent that answers question about
Alphabet stock prices (using get_stock_price tool)
and historical performance based on the data in Vertex AI Search datastore (using
doc_qa_agent tool)."""
)
```

2. 在終端機中，於 `financial_agent` 目錄內啟動 ADK 網頁介面，與代理互動：

```
adk web ~
```

3. 點選終端機輸出內容中提供的連結 (通常是 `http://127.0.0.1:8000`)，在瀏覽器中開啟 ADK 開發人員使用者介面。
-

11. 測試代理

現在您可以測試代理程式的推理能力，以及使用工具回答複雜問題的能力。

1. 在 ADK 開發人員使用者介面中，請務必從下拉式選單選取 `financial_agent`。
2. 請嘗試提出需要從美國證券交易委員會 (SEC) 歸檔資料 (非結構化資料) 取得資訊的問題。在對話中輸入下列查詢：

```
What were the total revenues for the quarter ending on March 31, 2023?
```

代理程式應呼叫 `search_and_qna_agent`，後者會使用 `VertexAiSearchTool` 在財務文件中尋找答案。

3. 接著，提出需要使用自訂工具 (結構化資料) 才能回答的問題。請注意，提示詞中的日期格式不一定要與函式規定的格式完全相符，LLM 會自動重新格式化。

```
What was the closing stock price for Alphabet on July 10, 2025?
```

代理程式應呼叫 `get_stock_price` 工具。點選對話中的工具圖示，即可檢查函式呼叫及其結果。

4. 最後，提出複雜問題，要求代理同時使用這兩項工具，並整合結果。

```
According to the 10-Q filing for the period ending March 31, 2023, what were the company's net cash provided by operating activities, and what was the stock's closing price on that date?
```

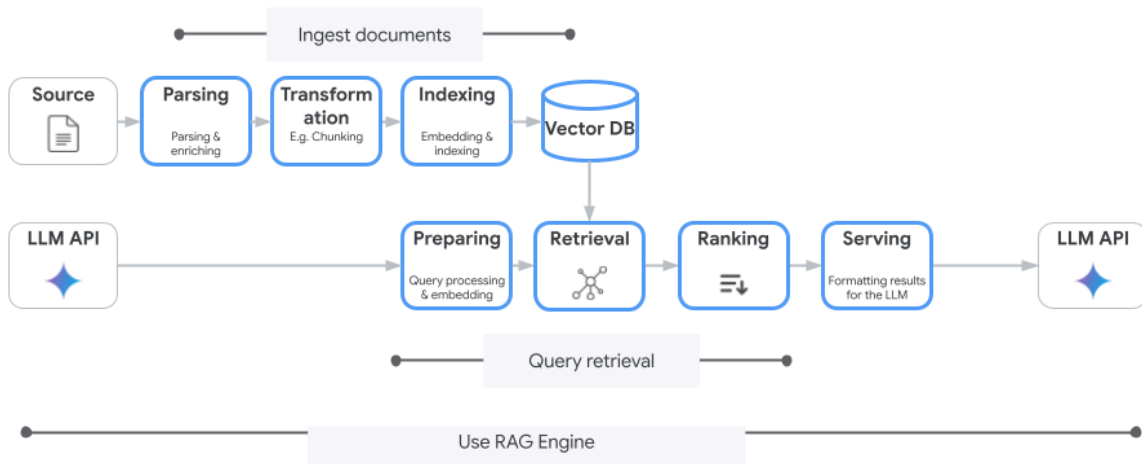
這項查詢會強制代理程式執行多步驟計畫：

- 首先，這個區塊會使用 `VertexAiSearchTool` 在 SEC 申報資料中尋找現金流量資訊。
- 接著，系統會辨識出需要股價，並使用日期 `2023-03-31` 呼叫 `get_stock_price` 函式。
- 最後，系統會將這兩項資訊合併為單一的完整答案。

5. 完成後，您可以關閉瀏覽器分頁，並在終端機中按下 `CTRL+C` 鍵，停止 ADK 伺服器。
-

12. 為工作選擇服務

[Vertex AI Search](#) 並非唯一可用的向量搜尋服務。您也可以使用代管服務，自動執行整個檢索增強生成流程：[Vertex AI RAG 引擎](#)。



從文件擷取到擷取和重新排序，全都由這項服務處理。RAG Engine 支援多個向量儲存空間，包括 [Pinecone](#) 和 [Weaviate](#)。

您也可以自行代管許多專用向量資料庫，或運用資料庫引擎中的向量索引功能，例如 PostgreSQL 服務中的 pgvector (例如 [AlloyDB](#) 或 [BigQuery 向量搜尋](#))。

支援向量搜尋的其他服務包括：

- [PostgreSQL 適用的 Cloud SQL](#)
- [MySQL 適用的 Cloud SQL](#)
- [Cloud Spanner](#)
- [Memorystore for Redis](#)
- [Firestore](#)
- [Bigtable](#)

選擇 Google Cloud 上的特定服務時，一般建議如下：

- 如果您已有運作良好且可妥善擴充的向量搜尋 DIY 基礎架構，請將其部署至 [Google Kubernetes Engine](#)，例如 [Weaviate](#) 或 [DIY PostgreSQL](#)。
- 如果資料位於 BigQuery、AlloyDB、Firestore 或任何其他資料庫，請考慮使用其向量搜尋功能，**在該資料庫中執行大規模語意搜尋，做為較大型查詢的一部分**。舉例來說，如果 BigQuery 資料表中有產品說明和/或圖片，加入文字和/或圖片嵌入欄位，就能大規模使用相似度搜尋功能。支援 ScANN 搜尋的向量索引可在索引中處理數十億個項目。
- 如要快速開始使用代管平台，並盡量減少工作量，請選擇 Vertex AI Search。這項全代管搜尋引擎和檢索器 API 非常適合複雜的企業用途，可提供高品質的現成功能、擴充性，以及精細的存取控管。這項功能可簡化連結各種企業資料來源的程序，並支援跨多個來源搜尋。
- 如果您想為開發人員尋找易用性和自訂功能之間的平衡點，請使用 Vertex AI RAG 引擎。可快速製作原型和開發，同時兼顧彈性。
- 探索[檢索增強生成參考架構](#)。

13. 結論

恭喜！您已成功建構並測試使用檢索增強生成技術的 AI 代理程式。您已學會如何：

- 運用 Vertex AI Search 強大的語意搜尋功能，為非結構化文件建立知識庫。
- 開發自訂 Python 函式，做為擷取結構化資料的工具。
- 使用 Agent Development Kit (ADK) 建立以 Gemini 為基礎的多工具代理。
- 建構可進行複雜多步驟推論的代理，回答需要整合多個來源資訊的查詢。

本實驗室會說明 Agentic RAG 的核心原則。這項強大的架構可在 Google Cloud 上建構智慧、準確且具備情境意識的 AI 應用程式。

從原型設計到投入正式環境

這個實驗室屬於「**Google Cloud 學習路徑：打造可用於正式環境的 AI**」。

- [探索完整課程](#)，從設計原型開始，一步步助您把專案投入正式環境。
 - 使用 #ProductionReadyAI 主題標記分享你的進度。
-